



MILESTONE 2 — DATA MODEL SPECIFICATION

FreightQuote AI

MongoDB Collections for Pricing, Margin & Approval — Weeks 3–4

DOCUMENT TYPE	Data model specification — M2 collections, document shapes, indexes, M1 relationships
PROJECT CODE	FQ-AMB-001 / M2-DATA
COMPANION TO	FreightQuote AI — Milestone 2 Execution Document
BUILDS ON	FreightQuote AI — Milestone 1 MongoDB Data Model (15 collections)
NEW COLLECTIONS	8
MODIFIED COLLECTIONS	4 — all additive, no migration required

Table of Contents

1. How M2 Attaches to M1

What carries over, what changes, what is new

2. Rate Card Collections

Cards, lines, surcharges, imports

3. Margin & Approval Collections

Policies, rules, approvals

4. Supporting Collections

FX rates

5. Changes to M1 Collections

Additive fields, why no migration is needed

6. Complete quote_versions Document

The full M2 shape end to end

7. Indexes

New indexes and what they serve

8. Transactions & Query Patterns

Multi-collection writes, dashboard reads

9. Validation & Setup Checklist

Schema validators and go-live checks

1. How M2 Attaches to M1

1.1 The 15 M1 collections

M1 collection	M2 status	What M2 does with it
users	Unchanged	Approval routing reads role
customers	Extended	Adds credit_profile for approval rule #5; tier now drives margin policy
ports	Unchanged	Surcharge applicability filters on port codes
port_distances	Unchanged	—
carriers	Unchanged	Rate cards reference carrier_id
carrier_services	Unchanged	—
container_types	Unchanged	Rate card lines key on container_type
business_calendars	Unchanged	Quote validity expiry respects it
lane_rates	Deprecated	Read-only for one release, then archived. See 5.4.
shipments	Unchanged	service.incoterm finally gets consumed
routes	Unchanged	Snapshotted into quote versions as before
freight_quotes	Extended	Adds approval and issuance state
quote_versions	Extended	The main change. pricing object fills out. See section 6.
counters	Unchanged	—
activity_logs	Unchanged	New action types: margin change, approval decision, issuance

1.2 Why no migration is required

In the M1 model, quote_versions.pricing was deliberately specified as a **nested object** rather than a flat set of fields:

```
// M1 shape — already an object
pricing: {
  is_indicative: true,
  lane_rate_ref: ObjectId,
  base: { amount: Decimal128, currency: "INR" },
  flat_uplift_pct: Decimal128("35.000"),
  final_quote: { amount: Decimal128, currency: "INR" },
  cost_components: [], // ← empty in M1, populated in M2
  total_cost: null, // ← null in M1, filled in M2
  margin_pct: null
}
```

M2 populates the empty fields and adds new keys inside the same object. Existing M1 documents remain valid and readable — they simply have is_indicative: true and an empty component array, which is an accurate description of what they are. No backfill, no downtime, no dual-read code path.

This is what the M1 design decision bought you

Had pricing been flattened to pricing_base, pricing_total, pricing_uplift, M2 would need eleven new top-level fields and a migration to move the old three. Nested objects absorb growth; flat field prefixes do not.

1.3 The 8 new M2 collections

Collection	Type	Purpose
rate_cards	Master	Carrier contract or spot sheet with validity window
rate_card_lines	Master	Per lane × container type (or weight break) base rate
surcharges	Master	BAF, THC, ISPS, PSS — with applicability rules
rate_card_imports	Operational	Two-phase import: validation report, then commit
margin_policies	Config	Floor, target, stretch by scope
approval_rules	Config	Ordered breach conditions → approver role
quote_approvals	Transactional	Approval requests and decisions
fx_rates	Master	Daily rates, frozen onto each quote version

2. Rate Card Collections

rate_cards

One document per carrier contract or spot sheet. Small collection, changes monthly.

```
{
  _id: ObjectId,
  name: "Maersk Contract 2026 H2 - Asia/ME",
  carrier_id: ObjectId, // → carriers (M1)
  carrier_name: "Maersk Line", // denormalised for list views
  type: "CONTRACT", // CONTRACT | SPOT | TARIFF
  customer_id: ObjectId | null, // set = customer-specific contract
  // null = general carrier card
  mode: "OCEAN", // OCEAN | AIR | GROUND_RAIL
  currency: "USD",

  valid_from: ISODate("2026-07-01"),
  valid_to: ISODate("2026-12-31"),

  status: "ACTIVE", // DRAFT|ACTIVE|EXPIRED|SUPERSEDED
  superseded_by_id: ObjectId | null,

  // provenance - needed when a broker disputes a rate
  source_file_key: "rate-cards/2026/maersk-h2.xlsx",
  import_id: ObjectId, // → rate_card_imports
  uploaded_by_id: ObjectId,
  line_count: 1284,

  created_at: ISODate,
  updated_at: ISODate
}
```

Indexes: {carrier_id:1, status:1, valid_from:1, valid_to:1} · {customer_id:1, status:1} · {status:1, valid_to:1} for the expiry sweep

rate_card_lines

The bulk collection — one document per lane x container type. Thousands of rows per card.

```
{
  _id: ObjectId,
  rate_card_id: ObjectId, // → rate_cards
  carrier_id: ObjectId, // denormalised - avoids a lookup on
  // the hottest query in M2

  // ---- lane ----
  origin_port_code: "INNSA",
  destination_port_code: "AEJEA",
  lane_key: "INNSA-AEJEA", // normalised, matches M1 convention

  // ---- OCEAN branch ----
  container_type: "40HC", // null on the air branch
  weight_break_min: null,

  // ---- AIR branch ----
  // container_type: null,
  // weight_break_min: 100, // kg - see M2 exec doc §4.3

  base_rate: { amount: Decimal128("1780.0000"), currency: "USD" },
  minimum_charge: { amount: Decimal128("1200.0000"), currency: "USD" },

  transit_days_quoted: 6, // carrier's own published transit
  is_active: true,
  created_at: ISODate
}
```

One collection, two shapes — and that is deliberate

Ocean lines carry container_type and null weight_break_min; air lines carry the reverse. Splitting into two collections would duplicate the lane, card and validity logic in every query. Keep one collection, enforce the branch with a \$jsonSchema oneOf, and let the resolver read mode from the parent card.

Indexes: {lane_key:1, container_type:1, rate_card_id:1} — the resolver's primary path · {rate_card_id:1} for bulk delete on supersede · {lane_key:1, weight_break_min:1} for the air path

surcharges

Independent of rate cards. Port-driven and route-driven, not carrier-driven.

```
{
  _id: ObjectId,
  code: "BAF", // matches the component catalogue
  name: "Bunker Adjustment Factor",
  description: "Fuel cost adjustment, revised monthly",
}
```

```

calculation_type: "PERCENT_OF_BASE", // FLAT_PER_SHIPMENT | PER_CONTAINER |
                                     // PER_KG | PER_RT | PERCENT_OF_BASE |
                                     // PERCENT_OF_VALUE | BASE_PLUS_PER_KM
value:      Decimal128("12.000"),    // 12% – unit depends on calculation_type
currency: null,                      // null for percentage types
per_km_rate: null,                   // BASE_PLUS_PER_KM only

// ---- applicability: empty array = applies to all ----
applies_to: {
  modes:      ["OCEAN"],
  carrier_ids: [],
  origin_ports: [],
  destination_ports: [],
  lane_keys:  [],
  container_types: [],
  cargo_flags: []                // ["HAZARDOUS"], ["REEFER"]
},

incoterm_leg: "FREIGHT",             // FREIGHT|ORIGIN|DESTINATION|HAULAGE|
                                     // SPECIAL|INSURANCE – drives $3 filtering
calculation_order: 20,               // see M2 exec doc §2.1

valid_from: ISODate("2026-08-01"),
valid_to:   ISODate("2026-08-31"),
is_active: true
}

```

On incoterm_leg: this is what lets the incoterm scope matrix filter surcharges generically instead of hardcoding component codes. A new surcharge added by an admin automatically lands in the right incoterm bucket.

Indexes: {code:1, valid_from:1, valid_to:1} · {is_active:1, "applies_to.modes":1} · {calculation_order:1}

rate_card_imports

The two-phase import. Validation writes here; commit reads the token back.

```

{
  _id:      ObjectId,
  file_key: "rate-cards/staging/2026-08-12-maersk.xlsx",
  file_name: "maersk-h2-2026.xlsx",
  carrier_id: ObjectId,
  uploaded_by_id: ObjectId,

  status: "VALIDATED", // UPLOADED|VALIDATING|VALIDATED|REJECTED|
                       // COMMITTED|FAILED

  validation_report: {
    rows_total: 1310,
    rows_parsed: 1284,
    rows_rejected: 26,
    hard_errors: 0, // > 0 blocks commit
    warnings: 3,
    issues: [
      { row: 47, severity: "ERROR",
        field: "origin_port_code", value: "INNSAA",
        message: "Unknown port code" },
      { row: 512, severity: "WARNING",
        field: "base_rate", value: "17800.00",
        message: "10x previous card rate – possible decimal error" },
      { row: 903, severity: "ERROR",
        field: "container_type", value: "45HC",
        message: "Unknown container type" }
    ],
    overlapping_cards: [
      { rate_card_id: ObjectId, name: "Maersk Contract 2026 H1",
        overlap_days: 14 }
    ]
  },

  validation_token: "vt_8f3a2c...", // required by the commit call
  expires_at:      ISODate,         // token valid 1 hour
  committed_rate_card_id: null,
  created_at:      ISODate
}

```

Indexes: {validation_token:1} · {status:1, created_at:-1} · TTL on expires_at

3. Margin & Approval Collections

margin_policies

Small config collection. Resolution is most-specific-wins across five scope levels.

```
{
  _id: ObjectId,
  scope: "LANE",          // GLOBAL | CARGO_TYPE | LANE |
                          // CUSTOMER_TIER | CUSTOMER_LANE
  scope_key: "INNSA-AEJEA",
  // GLOBAL      → scope_key: null
  // CARGO_TYPE  → "HAZARDOUS"
  // LANE        → "INNSA-AEJEA"
  // CUSTOMER_TIER → "STRATEGIC"
  // CUSTOMER_LANE → "|INNSA-AEJEA"

  resolution_priority: 3,    // 1 = most specific, 5 = GLOBAL

  floor_pct: Decimal128("12.000"),
  target_pct: Decimal128("15.000"),
  stretch_pct: Decimal128("19.000"),

  is_active: true,
  effective_from: ISODate,
  created_by_id: ObjectId,
  updated_at: ISODate
}
```

Indexes: {scope:1, scope_key:1, is_active:1} · {resolution_priority:1, is_active:1}

The resolver queries all candidate scopes in one call, sorts by `resolution_priority`, and takes the first. Storing the priority as a field rather than deriving it from scope keeps the ordering explicit and editable.

approval_rules

Ordered rules. All matching rules apply — a quote can require two approvals.

```
{
  _id: ObjectId,
  order_index: 1,
  name: "Deep discount – below floor by more than 5 points",

  condition: {
    type: "MARGIN_BELOW_FLOOR",
    // MARGIN_BELOW_FLOOR | QUOTE_VALUE_ABOVE |
    // HAS_PREDICTED_COMPONENT | NO_CREDIT_PROFILE |
    // RATE_CARD_EXPIRES_BEFORE_VALIDITY
    threshold: Decimal128("5.000"), // percentage points
    currency: null // set for QUOTE_VALUE_ABOVE
  },

  approver_role: "PRICING_MANAGER",
  is_blocking: true, // true = cannot issue without it
  is_active: true
}
```

Indexes: {order_index:1, is_active:1}

quote_approvals

One document per required approval. A quote version may have several.

```
{
  _id: ObjectId,
  quote_id: ObjectId, // → freight_quotes (M1)
  quote_version_id: ObjectId, // → quote_versions (M1)
  quote_number: "QT-2026-00934", // denormalised for the queue list
  customer_name: "Sharma Textiles Pvt Ltd",

  rule_id: ObjectId, // → approval_rules
  rule_name: "Deep discount – below floor by more than 5 points",
  breach_reason: "Margin 8.0% is 5.0 points below the lane floor of 13.0%",
  breach_detail: {
    applied_margin_pct: Decimal128("8.000"),
    floor_pct: Decimal128("13.000"),
    gap_points: Decimal128("5.000"),
    quote_value: { amount: Decimal128("398400.0000"), currency: "INR" }
  },

  requested_by_id: ObjectId,
  requested_by_name: "Ravi Kumar",
  requested_at: ISODate,

  approver_role: "PRICING_MANAGER",
  approver_id: ObjectId | null, // set on decision
}
```

```
    approver_name: null,

    decision:      "PENDING",           // PENDING|APPROVED|REJECTED
    comment:       null,                // mandatory on REJECTED
    decided_at:    null,

    is_blocking: true
}
```

Indexes: {approver_role:1, decision:1, requested_at:-1} — the approval queue · {quote_version_id:1} · {quote_id:1, decision:1}

Why breach_detail is stored rather than recomputed

The approver sees the quote days after it was submitted, by which time the margin policy may have been edited. Freezing the numbers that triggered the approval means the record explains itself permanently — the same reasoning that drives quote version immutability in M1.

4. Supporting Collections

fx_rates

One document per currency pair per day. Frozen onto each quote version.

```
{
  _id:      ObjectId,
  base:     "USD",
  quote:    "INR",
  rate:     Decimal128("83.240000"),
  rate_date: ISODate("2026-08-12"),
  source:   "ecb",                // ecb | manual | provider name
  fetched_at: ISODate
}
```

Indexes: {base:1, quote:1, rate_date:-1} · unique on {base:1, quote:1, rate_date:1, source:1}

Rate cards are frequently in USD while quotes are issued in INR. The rate used is copied onto the quote version, never referenced — a quote must reproduce identically when the rate has since moved.

5. Changes to M1 Collections

5.1 customers — add credit profile

```
// ADDED in M2
credit_profile: {
  status:      "APPROVED",    // NONE|PENDING|APPROVED|SUSPENDED
  limit:       { amount: Decimal128("2500000.0000"), currency: "INR" },
  terms:       "NET30",
  approved_at: ISODate,
  reviewed_by_id: ObjectId
}
```

Approval rule `N0_CREDIT_PROFILE` fires when `status` is `NONE` or `PENDING`. Existing customers without the field are treated as `NONE` — absent and `NONE` behave identically, so no backfill is needed.

5.2 freight_quotes — add approval and issuance state

```
// ADDED in M2
requires_approval: true,
approval_status:   "PENDING",  // NONE|PENDING|APPROVED|REJECTED
pending_approval_count: 1,
issued_at:         null,
valid_until:       null,
accepted_at:       null,
declined_at:       null,
decline_reason:    null,

// status enum EXTENDS — M1 values remain valid
// M1: DRAFT | ISSUED | ACCEPTED | DECLINED | EXPIRED
// M2 adds: PENDING_APPROVAL | APPROVED
```

5.3 quote_versions — the main change

Full shape in section 6. Summary of what fills in:

Field	M1	M2
pricing.is_indicative	true	false
pricing.cost_components[]	empty	Populated, ordered
pricing.total_cost	null	Sum of components
pricing.margin_*	null	Amount, pct, policy, suppression record
pricing.lane_rate_ref	set	null — replaced by <code>rate_card_id</code>
pricing.rate_card_id	—	New, frozen
pricing.incoterm_scope[]	—	New — which components were in scope
win_probability	—	New
document	—	New — PDF key and generation time

5.4 lane_rates — deprecation path

Stage	Action
M2 week 3	Stop writing. The resolver no longer reads it for new quotes.
M2 week 4 → M3	Keep read-only. M1 quote versions reference it via <code>lane_rate_ref</code> and must stay explainable.
After M3	Export to cold storage, drop the collection. Never delete while any quote version still references a row.

6. Complete quote_versions Document (M2)

```
{
  _id:      ObjectId,
  quote_id: ObjectId,
  quote_number: "QT-2026-00934",
  version:   2,

  // == FROM M1 - unchanged ==
  route_snapshot: {
    route_id: ObjectId, carrier_name: "Maersk Line", service_name: "MECL",
    legs: [ { seq:1, from:"INNSA", to:"AEJEA",
              transit_days: Decimal128("3.0"), is_transshipment:false } ],
    total_distance_nm: 1205,
    transit_days_min: 6, transit_days_max: 10,
    estimated_arrival: ISODate("2026-08-22"),
    transit_breakdown: { pickup: Decimal128("1.0"),
                        origin_dwell: Decimal128("3.0"),
                        linehaul: Decimal128("3.0"),
                        schedule_wait: Decimal128("3.5"),
                        dest_dwell: Decimal128("3.0"),
                        delivery: Decimal128("0.0") }
  },

  charge_basis: { type: "PER_CONTAINER", units: Decimal128("2"),
                  unit_label: "2 x 40HC",
                  actual_weight_kg: null, volumetric_weight_kg: null,
                  chargeable_weight_kg: null,
                  volume_cbm: null, weight_tonnes: null },

  currency:      "INR",
  fx_rate:        Decimal128("83.240000"),      // FROZEN
  fx_rate_date:   ISODate("2026-08-12"),        // FROZEN
  fx_rate_id:     ObjectId,                     // → fx_rates

  // == M2 - pricing fills out ==
  pricing: {
    is_indicative: false,                      // ← was true in M1
    lane_rate_ref: null,                       // ← M1 field, now unused

    // FROZEN rate card provenance
    rate_card_id:      ObjectId,
    rate_card_name:     "Maersk Contract 2026 H2 - Asia/ME",
    rate_card_rule:     "CONTRACT_CARRIER",    // which of the 5 priority
                                                // levels matched
    rate_card_expires: ISODate("2026-12-31"),

    // which incoterm legs were in scope
    incoterm:          "FOB",
    incoterm_scope:    ["PUH", "CCO", "THCO", "ISPS", "DOC"],

    // ordered cost build-up
    cost_components: [
      { order: 10, code: "OFR", name: "Ocean freight",
        calculation_type: "PER_CONTAINER",
        unit_rate: { amount: Decimal128("142407.4074"), currency: "INR" },
        units: Decimal128("2"),
        amount: { amount: Decimal128("284814.8148"), currency: "INR" },
        source: "RATE_CARD",                      // RATE_CARD|SURCHARGE_TABLE|
                                                // PREDICTED|MANUAL
        source_ref: ObjectId,                      // rate_card_line_id
        weight_break_used: null },

      { order: 20, code: "BAF", name: "Bunker adjustment factor",
        calculation_type: "PERCENT_OF_BASE",
        unit_rate: { amount: Decimal128("12.000"), currency: null },
        units: null,
        amount: { amount: Decimal128("34177.7778"), currency: "INR" },
        source: "SURCHARGE_TABLE", source_ref: ObjectId },

      { order: 30, code: "THCO", name: "Terminal handling - origin",
        calculation_type: "PER_CONTAINER",
        unit_rate: { amount: Decimal128("9250.0000"), currency: "INR" },
        units: Decimal128("2"),
        amount: { amount: Decimal128("18500.0000"), currency: "INR" },
        source: "SURCHARGE_TABLE", source_ref: ObjectId },

      { order: 32, code: "ISPS", name: "Security surcharge",
        amount: { amount: Decimal128("4200.0000"), currency: "INR" },
        source: "SURCHARGE_TABLE", source_ref: ObjectId },

      { order: 34, code: "DOC", name: "Documentation fee",
        calculation_type: "FLAT_PER_SHIPMENT",
        amount: { amount: Decimal128("3500.0000"), currency: "INR" },
        source: "SURCHARGE_TABLE", source_ref: ObjectId }
    ],

    has_predicted_component: false,
    total_cost: { amount: Decimal128("345192.5926"), currency: "INR" },
  }
}
```

```

// margin
margin_policy_id: ObjectId,
margin_policy_scope: "LANE",
floor_pct: Decimal128("12.000"),
target_pct: Decimal128("15.000"),
applied_margin_pct: Decimal128("14.200"),
margin_amount: { amount: Decimal128("49017.3481"), currency: "INR" },

// agent recommendation vs what was applied
recommended_margin_pct: Decimal128("11.000"),
margin_suppressed: true,
suppression_reason: "Agent recommendation 11.0% below lane floor 12.0%",

final_quote: { amount: Decimal128("394209.9407"), currency: "INR" }
},

win_probability: Decimal128("0.6200"),
win_probability_model: "winprob-v1.0-baseline",

valid_until: ISODate("2026-08-19T23:59:59Z"),

document: {
  pdf_key: "quotes/2026/QT-2026-00934-v2.pdf",
  generated_at: ISODate,
  template_version: "quote-tpl-v1.2"
},

model_versions: { transit: "transit-v1.2-lgbm",
  route_scorer: "rules-v1.0",
  win_probability: "winprob-v1.0-baseline" },
data_as_of: { carrier_schedules: ISODate, port_congestion: ISODate,
  port_distances: ISODate, fx: ISODate,
  rate_card: ISODate },
assumptions: [
  "Rate valid for the stated container type and count only",
  "Excludes duties, taxes and customs clearance",
  "Subject to space and equipment availability",
  "Rate card valid to 31 Dec 2026"
],
rationale: {
  route: "Direct weekly service, composite score 0.86...",
  transit: "13.5 days computed; presented as a 6-10 day range...",
  pricing: "Maersk contract card matched at CONTRACT_CARRIER priority...",
  margin: "Lane floor 12.0% applied over agent recommendation of 11.0%..."
},

status: "APPROVED", // DRAFT|PENDING_APPROVAL|APPROVED|
// ISSUED|SUPERSEDED
created_by_id: ObjectId,
created_at: ISODate
}

```

7. Indexes

```
// == NEW COLLECTIONS ==
rate_cards      { carrier_id:1, status:1, valid_from:1, valid_to:1 }
                { customer_id:1, status:1 }
                { status:1, valid_to:1 }

rate_card_lines { lane_key:1, container_type:1, rate_card_id:1 } * hot path
                { lane_key:1, weight_break_min:1 }             air path
                { rate_card_id:1 }                             bulk ops

surcharges      { code:1, valid_from:1, valid_to:1 }
                { is_active:1, "applies_to.modes":1 }
                { calculation_order:1 }

rate_card_imports { validation_token:1 }
                  { status:1, created_at:-1 }
                  { expires_at:1 } TTL

margin_policies  { scope:1, scope_key:1, is_active:1 }
                  { resolution_priority:1, is_active:1 }

approval_rules  { order_index:1, is_active:1 }

quote_approvals { approver_role:1, decision:1, requested_at:-1 } * queue
                  { quote_version_id:1 }
                  { quote_id:1, decision:1 }

fx_rates        { base:1, quote:1, rate_date:-1 }
                  { base:1, quote:1, rate_date:1, source:1 } unique

// == ADDED TO M1 COLLECTIONS ==
freight_quotes  { approval_status:1, created_at:-1 }
                  { status:1, valid_until:1 }             expiry sweep

quote_versions  { "pricing.applied_margin_pct":1 } margin analytics
                  { "pricing.rate_card_id":1 }             impact analysis
                  { "pricing.has_predicted_component":1, status:1 }
```

The starred two carry the load. `rate_card_lines` is queried on every quote generation, and the approval queue is polled by every approver's browser. Verify both with `explain()` showing IXSCAN before the milestone review.

8. Transactions & Query Patterns

8.1 Multi-collection writes

Operation	Collections touched
Commit a rate card import	Insert rate_cards, bulk-insert rate_card_lines, set overlapping card to SUPERSEDED, update rate_card_imports. One transaction — a half-imported card prices quotes wrongly.
Create a quote version	Insert quote_versions, set previous to SUPERSEDED, update freight_quotes.current_version_id. Transaction.
Submit for approval	Insert one or more quote_approvals, update quote status and pending_approval_count. Transaction.
Approve	Update the approval, decrement the counter, and if it reaches zero set the quote to APPROVED. Transaction — a race here approves a quote that still has a blocking approval outstanding.
Adjust margin	Insert a new quote version. Never update the existing one.
Expiry sweep	updateMany on issued quotes past valid_until. Idempotent, no transaction.

8.2 Query patterns

Screen or job	Shape
Rate card resolution	One find on rate_card_lines joined in application code against active cards, sorted by the 5-level priority. Cache the active card list in Redis — it changes monthly, not per request.
Surcharge resolution	find on active surcharges for the mode, then filter applicability in Python. The collection is small; do not push complex \$or arrays into Mongo.
Margin policy resolution	One find on all candidate scope keys, sort by resolution_priority, take first.
Approval queue	find({approver_role, decision:"PENDING"}).sort({requested_at:1}) — oldest first, so nothing sits forgotten.
Quote builder	One findOne on the version. All components, margin and rationale are embedded — the whole screen renders from one document.
Margin analysis	aggregate grouping versions by lane_key. Read replica. Pre-computed rollups in M4.

9. Validation & Setup Checklist

9.1 Schema validator — the branch that matters

```
db.createCollection("rate_card_lines", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["rate_card_id", "lane_key", "base_rate", "is_active"],
    properties: {
      lane_key: { bsonType: "string", pattern: "^[A-Z]{3,5}-[A-Z]{3,5}$" },
      base_rate: { bsonType: "object", required: ["amount", "currency"],
        properties: { amount: { bsonType: "decimal" } } },
      minimum_charge: { bsonType: "object",
        properties: { amount: { bsonType: "decimal" } } }
    }
  },
  oneOf: [
    { required: ["container_type"],
      properties: { weight_break_min: { bsonType: "null" } } },
    { required: ["weight_break_min"],
      properties: { container_type: { bsonType: "null" } } }
  ]
}},
validationLevel: "strict", validationAction: "error"
})
```

The oneOf enforces the ocean/air branch at the database level, so a malformed import cannot create a line that is neither.

9.2 Checklist

☐	Item
☐	All 8 new collections created with \$jsonSchema validators, validationAction: "error"
☐	Every monetary field declared bsonType: "decimal"; a CI test scans for any double in an amount field
☐	rate_card_lines oneOf branch enforced; a line with both or neither is rejected
☐	Import commit runs in a transaction; a forced mid-import failure leaves no partial card
☐	Quote version immutability test: updating an existing version fails except for status
☐	Approval counter test: two approvers deciding simultaneously cannot both set APPROVED while a blocking approval is outstanding
☐	Margin floor test: below-floor writes rejected at the service layer with a suppression record written
☐	Portal projection test: no cost_components, total_cost, margin_* or win_probability key in any customer response
☐	Reproducibility test: a sample of issued quotes regenerates identically from frozen inputs
☐	All indexes in section 7 created and verified with explain() showing IXSCAN
☐	lane_rates set read-only, not dropped — M1 quotes still reference it
☐	Seed script loads margin policies, approval rules and at least one real rate card